

Induced Metric Optimisers

March 16, 2026

Contents

1	Introduction	3
2	Setup and Notation	3
3	Fixed Diagonal Metric	4
3.1	Geometric Derivation	4
3.2	Practical Algorithm	5
3.3	Log-Loss Variant	6
3.4	Implementation	6
4	Learnable Scalar Metric	6
4.1	Construction	7
4.2	Metric Learning	7
4.3	Algorithm	8
4.4	Hyperparameters	8
4.5	Implementation	8
5	Learnable Diagonal Metric	9
5.1	Construction	9
5.2	Metric Learning	9
5.3	Mean-Centering	10
5.4	Algorithm	11
5.5	Implementation	11
6	Off-Diagonal Metric	11
6.1	Derivation	11
6.2	Algorithm	13
6.3	Implementation	13
7	Shared Implementation Details	14
7.1	Update Direction vs. Metric Direction	14
7.2	Computational Cost	14
7.3	Shared Hyperparameters	15
8	Results	15

A	Repository Structure	15
A.1	Directory Layout	15
A.2	Running Sweeps	16
A.3	Available Optimisers	16
A.4	Analysis Notebooks	16

1 Introduction

Harvey [1] introduced a class of optimisers that precondition gradients using the Riemannian metric naturally induced when the loss landscape is embedded in higher-dimensional space. The key idea is to embed the N -dimensional parameter space into $\mathbb{R}^{N+1}$ via the loss function, equip the ambient space with a metric h , and pull h back to obtain a geometry-aware preconditioner. The resulting update automatically damps the effective learning rate in high-curvature regions—acting as a smooth form of gradient clipping—while adding only $O(N)$ overhead.

This document extends that framework in three directions (summarised in Table 1):

- (i) **Learnable scalar metric** (Section 4): the inverse base metric γ^{-1} is parameterised by a single learnable scalar s , allowing the overall metric scale to adapt during training.
- (ii) **Learnable diagonal metric** (Section 5): each parameter receives its own learnable scale s_i , enabling per-parameter metric adaptation with a principled gauge-fixing (mean-centering) to remove the degeneracy with the global scaling hyperparameter ξ .
- (iii) **Off-diagonal ambient metric** (Section 6): the ambient metric h in [1] has no coupling between the parameter dimensions and the loss dimension (the off-diagonal blocks are zero). We relax this by introducing coupling vectors $\mathbf{a}$ and $\mathbf{b}$, which add a second component to the update direction and enable geometry-induced momentum, weight decay, and other behaviours.

Table 1: Summary of optimiser variants. All share the same geometric construction (embed, pull back, invert via Sherman-Morrison-Woodbury) and are $O(N)$ per step. Sections 4–6 are the new contributions of this document.

Variant	Base metric γ^{ij}	Learnable?	Section
Fixed diagonal	$\gamma \delta^{ij}$ (hyperparameter)	No	3
Learnable scalar	$e^s \delta^{ij}$	Yes ($s \in \mathbb{R}$)	4
Learnable diagonal	$e^{s_i} \delta^{ij}$	Yes ($\mathbf{s} \in \mathbb{R}^N$)	5
Off-diagonal	$\gamma \delta^{ij} + \text{off-diag. } \mathbf{a}, \mathbf{b}$	No	6

Roadmap. Section 2 establishes notation. Section 3 reproduces the core method of [1], including the full algorithms, and defines the practical machinery (momentum, EMA, log-loss variant) used by all subsequent sections. The three extensions follow in Sections 4–6, each with its own algorithm. Section 7 collects implementation details common to all variants, and Section 8 presents benchmark results. Appendix A documents the repository structure and sweep infrastructure.

A note on what is being learned. In the learnable variants (Sections 4 and 5), we parameterise and learn the *inverse* base metric γ^{ij} , not the metric γ_{ij} itself. This is natural because the update rule requires γ^{-1} , and parameterising the inverse directly avoids an inversion step.

2 Setup and Notation

Given a model with N parameters $\theta(t) = \{\theta^i(t)\}_{i=1}^N$ and a loss function $\mathcal{L} : \mathbb{R}^N \rightarrow \mathbb{R}$, our goal is to find the optimal parameter update $\delta\theta$ at each time step t . That is, in preconditioned gradient flow we have

$$\frac{d\theta^i}{dt} = -g^{ij} \frac{\partial \mathcal{L}}{\partial \theta^j} \implies \delta\theta^i \approx -\eta g^{ij} \frac{\partial \mathcal{L}}{\partial \theta^j}$$

where $\eta \in \mathbb{R}_{>0}$ is the learning rate and g^{ij} is the inverse of the induced metric on the parameter space.

Across all variants in this document the induced metric is obtained by the same geometric construction: we embed parameter space into an ambient space via

$$\phi : \mathbb{R}^N \rightarrow \mathbb{R}^{N+1}, \quad \phi(\theta^1, \dots, \theta^N) = (\theta^1, \dots, \theta^N, f(\theta)),$$

where $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is a scalar function of the parameters (taken to be the loss $\mathcal{L}$, or a function thereof), and then pull back an ambient bilinear form $h \in \mathbb{R}^{(N+1) \times (N+1)}$ to the parameter space. The variants differ in the choice of h (and, for the log-loss variant, the embedding function f ; see below).

We use the shorthand $l_i := \partial \mathcal{L} / \partial \theta^i$ for the raw gradient. When a base metric γ_{ij} is present, we write $l^i := \gamma^{ij} l_j$ for the metric-raised gradient (i.e. the inverse metric applied to $\mathbf{l}$). When $\gamma = \mathbb{1}_N$ (the identity), raising an index has no effect, so $l^i = l_i$ and the two notations coincide. The same index-raising convention applies to any covector: $a^i := \gamma^{ij} a_j$, $m^i := \gamma^{ij} m_j$, etc.

3 Fixed Diagonal Metric

This section reproduces the core method of [1]: a fixed base metric γ defines the ambient space geometry, and the induced metric on parameter space provides a preconditioner for gradient descent. The geometric derivation, practical algorithms, and all notation established here carry forward to the extensions in Sections 4–6.

3.1 Geometric Derivation

The original ambient metric in [1] is

$$h = \begin{pmatrix} \gamma & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1) \times (N+1)}$$

where $\gamma \in \mathbb{R}^{N \times N}$ is symmetric positive-definite (taken to be diagonal or scalar in all our experiments). The induced metric g on the parameter space is given by

$$g_{ij} = h_{\alpha\beta} \frac{\partial \phi^\alpha}{\partial \theta^i} \frac{\partial \phi^\beta}{\partial \theta^j} = \gamma_{ij} + \frac{\partial \mathcal{L}}{\partial \theta^i} \frac{\partial \mathcal{L}}{\partial \theta^j}.$$

Then, to calculate the parameter updates $\delta \theta^i$ we need g^{ij} , the inverse of g_{ij} . We can compute this inverse using the Sherman-Morrison-Woodbury formula:

Sherman-Morrison-Woodbury Formula

Let $A \in \mathbb{R}^{n \times n}$ be an invertible matrix, and let $U, V \in \mathbb{R}^{n \times k}$ be matrices. Then, if $C = A + UV^\top$ is invertible, we have

$$C^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1}U)^{-1}V^\top A^{-1}.$$

This gives us

$$g^{ij} = \gamma^{ij} - \frac{\gamma^{ik} \gamma^{jl} \frac{\partial \mathcal{L}}{\partial \theta^k} \frac{\partial \mathcal{L}}{\partial \theta^l}}{1 + \gamma^{mn} \frac{\partial \mathcal{L}}{\partial \theta^m} \frac{\partial \mathcal{L}}{\partial \theta^n}} = \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k}$$

where

$$l_i := \frac{\partial \mathcal{L}}{\partial \theta^i} \implies l^i = \gamma^{ij} l_j.$$

Thus, the parameter updates are given by

$$\begin{aligned}\delta\theta^i &= -\eta \left(\gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k} \right) l_j \\ &= -\eta l^i \left(1 - \frac{l_j l^j}{1 + l_k l^k} \right) \\ &= -\eta l^i \left(\frac{1}{1 + l_k l^k} \right).\end{aligned}$$

So to summarize, (1) we start with an ambient metric h which induces a metric g on the parameter space, (2) we compute the inverse g^{ij} , and (3) we use g^{ij} to compute the parameter updates $\delta\theta^i$.

3.2 Practical Algorithm

The theoretical update above becomes a practical optimiser by adding a hyperparameter ξ to control the metric strength, momentum with bias correction, an exponential moving average (EMA) for the denominator, and decoupled weight decay. The full procedure is given in Algorithm 1 (adapted from Algorithm 1 of [1]). The log-loss variant is Algorithm 2 (adapted from Algorithm 2 of [1]).

Algorithm 1 Fixed diagonal metric (plain embedding, $f = \mathcal{L}$)

Require: learning rate η , momentum decay β_m , metric strength ξ , denominator EMA decay β , weight decay λ_θ , inverse base metric γ^{-1}

```

1: Initialise  $t \leftarrow 0$ ,  $\mathbf{m}_0 \leftarrow \mathbf{0}$ ,  $\bar{v}_0 \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $\mathbf{l}_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_{t-1})$ 
5:    $\mathbf{v}_t \leftarrow \xi \cdot \sum_i \mathbf{l}_{t,i} (\gamma^{-1} \mathbf{l}_t)_i$  // trace statistic
6:    $\bar{\mathbf{v}}_t \leftarrow \beta \bar{\mathbf{v}}_{t-1} + (1 - \beta) \mathbf{v}_t$  // denominator EMA
7:    $\hat{\mathbf{v}}_t \leftarrow \bar{\mathbf{v}}_t / (1 - \beta^t)$  // bias correction
8:    $\mathbf{r}_t \leftarrow 1 / (1 + |\hat{\mathbf{v}}_t|)$  // damping factor
9:    $\mathbf{m}_t \leftarrow \beta_m \mathbf{m}_{t-1} + (1 - \beta_m) \mathbf{l}_t$  // momentum
10:   $\hat{\mathbf{m}}_t \leftarrow \gamma^{-1} \mathbf{m}_t / (1 - \beta_m^t)$  // bias-corrected, metric-raised
11:   $\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{r}_t \hat{\mathbf{m}}_t - \eta \lambda_\theta \theta_{t-1}$ 
12: end while
13: return  $\theta_t$ 
```

Algorithm 2 Fixed diagonal metric (log-loss embedding, $f = \log \mathcal{L}$, requires $\mathcal{L} > 0$)

Require: same as Algorithm 1

```

1: Initialise  $t \leftarrow 0$ ,  $\mathbf{m}_0 \leftarrow \mathbf{0}$ ,  $\bar{v}_0 \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $\mathbf{l}_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_{t-1})$ ;  $L_t \leftarrow \mathcal{L}(\theta_{t-1})$ 
5:    $\mathbf{v}_t \leftarrow \xi \cdot \sum_i \mathbf{l}_{t,i} (\gamma^{-1} \mathbf{l}_t)_i$ 
6:    $\bar{\mathbf{v}}_t \leftarrow \beta \bar{\mathbf{v}}_{t-1} + (1 - \beta) \mathbf{v}_t$ ;  $\hat{\mathbf{v}}_t \leftarrow \bar{\mathbf{v}}_t / (1 - \beta^t)$ 
7:    $\mathbf{r}_t \leftarrow L_t / (L_t^2 + |\hat{\mathbf{v}}_t|)$  // log-loss damping
8:    $\mathbf{m}_t \leftarrow \beta_m \mathbf{m}_{t-1} + (1 - \beta_m) \mathbf{l}_t$ ;  $\hat{\mathbf{m}}_t \leftarrow \gamma^{-1} \mathbf{m}_t / (1 - \beta_m^t)$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \eta \mathbf{r}_t \hat{\mathbf{m}}_t - \eta \lambda_\theta \theta_{t-1}$ 
10: end while
11: return  $\theta_t$ 
```

In short, the practical update at each step is

$$\delta\theta^i = -\eta r_t \hat{m}^i, \quad r_t = \frac{1}{1 + |\hat{v}_t|}, \quad v_t = \xi \gamma^{mn} l_{t,m} l_{t,n}, \quad (1)$$

where $\hat{v}_t$ is the bias-corrected EMA of v_t (lines 6–8 of Algorithm 1) and $\hat{m}^i$ is the bias-corrected, metric-raised momentum (line 10).

3.3 Log-Loss Variant

Log-Loss Derivation Sketch

With $f = \log \mathcal{L}$ the embedding gradient is $\partial f / \partial \theta^i = l_i / \mathcal{L}$, so the induced metric becomes $g_{ij} = \gamma_{ij} + \xi (l_i / \mathcal{L})(l_j / \mathcal{L})$. Applying Sherman-Morrison-Woodbury gives the inverse metric

$$g^{ij} = \gamma^{ij} - \frac{\xi l^i l^j}{\mathcal{L}^2 + \xi l_k l^k}.$$

Contracting with the *raw* gradient l_j yields $g^{ij} l_j = l^i \cdot \mathcal{L}^2 / (\mathcal{L}^2 + v_t)$, where $v_t = \xi \gamma^{mn} l_m l_n$.

The full geometric update, however, uses the embedding gradient $\partial_j f = l_j / \mathcal{L}$, not the raw gradient:

$$g^{ij} \frac{\partial f}{\partial \theta^j} = \frac{g^{ij} l_j}{\mathcal{L}} = l^i \cdot \frac{\mathcal{L}}{\mathcal{L}^2 + v_t}.$$

The practical damping factor $r_t = \mathcal{L}_t / (\mathcal{L}_t^2 + |\hat{v}_t|)$ (line 7 of Algorithm 2) therefore follows directly from the geometry of the log embedding. This provides stronger damping than $\mathcal{L}^2 / (\mathcal{L}^2 + |\hat{v}_t|)$ as $\mathcal{L} \rightarrow 0$, preventing overshooting near minima.

In practice, the update direction $\mathbf{u}$ and the metric direction $\mathbf{l}$ need not coincide; see Section 7 for the full treatment.

3.4 Implementation

- JAX/Optax: `optimisers/jax_fixed.py`
- PyTorch: `optimisers/torch_fixed.py`
- Equivalence check: `optimisers/compare_fixed.py`

Exposed optimisers: `custom_sgd` (plain embedding, $f = \mathcal{L}$), `custom_sgd_log` (log-loss embedding, $f = \log \mathcal{L}$; requires $\mathcal{L} > 0$), `custom_sgd_rms` (RMS-normalised gradients, i.e. the update direction is $l_i / \|\mathbf{l}\|$ instead of l_i). The sweep name is `sgd_metric` / `sgd_log_metric` / `sgd_rms`.

4 Learnable Scalar Metric

The fixed diagonal variant treats γ as a hyperparameter chosen before training. A natural next step is to let the metric adapt during optimisation. We begin with the simplest case: a single learnable scalar that controls the global scale of γ^{-1} .

4.1 Construction

We parameterise the inverse metric as

$$\gamma^{ij} = \alpha \delta^{ij}, \quad \alpha = e^s,$$

where $s \in \mathbb{R}$ is a single learnable scalar. (We learn the *inverse* metric directly because the update rule requires γ^{-1} .) Initialising $s = 0$ gives $\gamma^{-1} = \mathbb{1}_N$, so the optimiser starts as the unscaled fixed diagonal variant.

The ambient metric remains block-diagonal,

$$h = \begin{pmatrix} e^{-s} \mathbb{1}_N & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s} \delta_{ij} + l_i l_j.$$

Applying the Sherman-Morrison-Woodbury formula with $A_{ij} = e^{-s} \delta_{ij}$, $u = v = \mathbf{l}$:

$$g^{ij} = e^s \delta^{ij} - \frac{l^i l^j}{1 + l_k l^k}.$$

Here $l^i = \gamma^{ij} l_j = e^s l_i$, so $l^i l^j = e^{2s} l_i l_j$ and $l_k l^k = e^s \|\mathbf{l}\|^2$. The parameter update is therefore

$$\delta \theta^i = -\eta g^{ij} l_j = -\eta l^i \left(1 - \frac{l_j l^j}{1 + l_k l^k} \right) = -\frac{\eta l^i}{1 + l_k l^k}.$$

Since $l_k l^k = e^s \|\mathbf{l}\|^2$, the denominator depends on s through the metric. The practical update follows the same structure as (1), with $\gamma^{ij} = \alpha \delta^{ij}$ and the trace statistic specialised to

$$v_t = \xi \alpha \|\mathbf{l}_t\|^2. \tag{2}$$

4.2 Metric Learning

We want to learn s so that $\alpha = e^s$ is calibrated to the local geometry of the loss. The trace statistic $v_t = \xi \alpha \|\mathbf{l}_t\|^2$ is the quantity that controls the denominator damping in the update rule: it measures how strongly the gradient stretches the embedded loss surface under the current metric. The only scalar that couples γ to the gradient $\mathbf{l}$ is v_t itself, making it the natural surrogate objective for adapting s .

We perform *gradient ascent* on v_t with respect to s :

$$\frac{\partial v_t}{\partial s} = \xi e^s \|\mathbf{l}_t\|^2 = v_t.$$

Why ascent? Increasing s increases $\alpha = e^s$, which amplifies the preconditioned direction $\alpha \hat{\mathbf{m}}$ in the numerator. At the same time, v_t (and hence $\hat{v}_t$) grows, which shrinks the damping factor $r_t = 1/(1 + |\hat{v}_t|)$ in the denominator. These two effects partially cancel: the metric becomes more responsive to the gradient structure, but the self-regulating denominator prevents the step size from blowing up. Gradient *descent* on v_t would push $s \rightarrow -\infty$, collapsing the metric towards zero and making the optimiser ignore the geometry entirely. Ascent keeps the metric “engaged” with the loss landscape.

Regularisation and stability. Note that $v_t \geq 0$, so the unregularised ascent rule monotonically increases s . Left unchecked this would eventually saturate the clipping bound. Three mechanisms prevent this: (i) an L2 regularisation term $-\mu\lambda s$ that provides a restoring force towards $s = 0$ (i.e. $\gamma^{-1} = \mathbb{1}$), (ii) hard clipping of s to $[-s_{\max}, s_{\max}]$, and (iii) the denominator EMA itself, which smooths transient gradient spikes. The resulting dynamics should be viewed as a stabilised feedback controller rather than an unbiased estimator of an optimal metric parameter.

The online metric update is

$$s \leftarrow s + \mu v_t - \mu \lambda s, \quad (3)$$

where $\mu > 0$ is the metric learning rate and $\lambda > 0$ is the L2 regularisation coefficient, followed by clipping to $[-s_{\max}, s_{\max}]$.

4.3 Algorithm

Algorithm 3 Learnable scalar metric (plain embedding)

Require: all hyperparameters from Algorithm 1, plus metric learning rate μ , metric regularisation λ , clipping bound $s_{\max}$

```

1: Initialise  $t \leftarrow 0$ ,  $\mathbf{m}_0 \leftarrow \mathbf{0}$ ,  $\bar{v}_0 \leftarrow 0$ ,  $s \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ ;  $\alpha \leftarrow e^s$ 
4:    $\mathbf{l}_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_{t-1})$ 
5:    $v_t \leftarrow \xi \alpha \|\mathbf{l}_t\|^2$ 
6:    $\bar{v}_t \leftarrow \beta \bar{v}_{t-1} + (1 - \beta) v_t$ ;  $\hat{v}_t \leftarrow \bar{v}_t / (1 - \beta^t)$ ;  $r_t \leftarrow 1 / (1 + |\hat{v}_t|)$ 
7:    $\mathbf{m}_t \leftarrow \beta_m \mathbf{m}_{t-1} + (1 - \beta_m) \mathbf{l}_t$ ;  $\hat{\mathbf{m}}_t \leftarrow \alpha \mathbf{m}_t / (1 - \beta_m^t)$ 
8:    $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{\mathbf{m}}_t - \eta \lambda_{\theta} \theta_{t-1}$  // parameter update
9:    $s \leftarrow s + \mu v_t - \mu \lambda s$  // metric update
10:   $s \leftarrow \text{clip}(s, -s_{\max}, s_{\max})$ 
11: end while
12: return  $\theta_t$ 
```

The log-loss variant replaces line 6 with $r_t \leftarrow L_t / (L_t^2 + |\hat{v}_t|)$.

4.4 Hyperparameters

The learnable scalar variant introduces three new hyperparameters beyond those of the fixed diagonal variant:

- μ (**metric_lr**): step size for the online metric update in (3).
- λ (**metric_reg**): L2 regularisation strength on s , controlling how far the metric drifts from identity.
- $s_{\max}$ (**metric_clip**): clipping bound on s , bounding $\alpha \in [e^{-s_{\max}}, e^{s_{\max}}]$.

4.5 Implementation

- JAX/Optax: `optimisers/jax_learnable_scalar.py`
- PyTorch: `optimisers/torch_learnable_scalar.py`
- Equivalence check: `optimisers/compare_learnable_scalar.py`

The plain variant is `custom_sgd_learnable_scalar` (sweep: `sgd_learn_scalar`). The log-loss variant is `custom_sgd_log_learnable_scalar` (sweep: `sgd_learn_scalar_log`).

5 Learnable Diagonal Metric

The learnable scalar metric has a single degree of freedom and therefore applies the same scale to every parameter. We now generalise to a per-parameter learnable scale, giving each parameter its own adaptive metric entry.

5.1 Construction

We promote s to a full vector $\mathbf{s} \in \mathbb{R}^N$ with one entry per parameter, giving the per-parameter inverse metric

$$\gamma^{ij} = e^{s_i} \delta^{ij}.$$

The ambient metric is

$$h = \begin{pmatrix} \text{diag}(e^{-\mathbf{s}}) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s_i} \delta_{ij} + l_i l_j.$$

This is again a rank-1 update to a diagonal matrix. Applying Sherman-Morrison-Woodbury with $A_{ij} = e^{-s_i} \delta_{ij}$ and $u = v = \mathbf{l}$,

$$g^{ij} = e^{s_i} \delta^{ij} - \frac{l^i l^j}{1 + l_k l^k},$$

where $l^i = \gamma^{ij} l_j = e^{s_i} l_i$ (no sum on i), so $l^i l^j = e^{s_i} l_i \cdot e^{s_j} l_j$ and $l_k l^k = \sum_k e^{s_k} l_k^2$. The update is

$$\delta \theta^i = -\eta g^{ij} l_j = -\eta \left(l^i - \frac{l^i (l_k l^k)}{1 + l_k l^k} \right) = -\frac{\eta l^i}{1 + l_k l^k}.$$

With ξ , momentum, and the EMA damping, the practical update is

$$\delta \theta^i = -\eta r_t \hat{m}^i, \quad r_t = \frac{1}{1 + |\hat{v}_t|}, \quad v_t = \xi \sum_k e^{s_k} l_k^2, \quad (4)$$

with the same log-loss variant as before. Compared to the scalar case, each parameter θ^i now has an individual scale e^{s_i} , allowing the metric to adapt separately to different regions of parameter space.

5.2 Metric Learning

The per-parameter metric update treats each s_i independently. We use the same surrogate-ascent strategy as in Section 4.2, now applied to the trace statistic $v_t = \xi \sum_k e^{s_k} l_k^2$ which serves as the surrogate objective:

$$J(\mathbf{s}) = v_t = \xi \sum_k e^{s_k} l_k^2.$$

This is the natural choice: v_t is the scalar that appears in the denominator of the update rule and it is the only quantity that couples all of γ , $\mathbf{l}$, and $\mathbf{s}$. Its gradient with respect to s_i is

$$\frac{\partial J}{\partial s_i} = \xi e^{s_i} l_i^2.$$

The online update is therefore

$$s_i \leftarrow s_i + \mu \xi e^{s_i} l_i^2 - \mu \lambda s_i, \quad (5)$$

followed by mean-centering and clipping (discussed below). The motivation for gradient *ascent* (not descent) and the role of regularisation are the same as in Section 4.2.

5.3 Mean-Centering

Parameter tensors and leaves. In frameworks like JAX and PyTorch, a model’s parameters are stored as a collection of tensors—for example, one weight matrix and one bias vector per layer. Each such tensor is called a *leaf* of the parameter tree. We write N_ℓ for the number of scalar entries in a single leaf (e.g. $N_\ell = d_{\text{in}} \times d_{\text{out}}$ for a weight matrix), as distinct from the total parameter count $N = \sum_\ell N_\ell$.

The centering operation. After the gradient step (5), we apply a per-leaf mean-centering operation:

$$s_i \leftarrow s_i - \bar{s}, \quad \bar{s} = \frac{1}{N_\ell} \sum_{k=1}^{N_\ell} s_k.$$

This is not an ad-hoc trick but a principled normalisation arising from a gauge degeneracy.

Gauge degeneracy. Write each log-scale as a mean plus a deviation: $s_k = \bar{s} + \delta_k$. By construction $\sum_k \delta_k = \sum_k (s_k - \bar{s}) = N_\ell \bar{s} - N_\ell \bar{s} = 0$. The trace statistic then factorises as

$$v = \xi \sum_k e^{s_k} l_k^2 = \xi e^{\bar{s}} \sum_k e^{\delta_k} l_k^2.$$

The global mean $\bar{s}$ acts as a multiplicative prefactor that is exactly degenerate with ξ : shifting all s_k by a constant c is equivalent to scaling ξ by e^c and leaving $\mathbf{s}$ unchanged. Mean-centering enforces $\bar{s} = 0$ (note: subsequent clipping may re-introduce a small nonzero mean), which fixes this gauge and ensures that ξ remains the single scalar controlling the overall metric magnitude while $\mathbf{s}$ captures only the *relative* per-parameter geometry. Since we apply this per leaf rather than globally, a separate mean mode is removed in each parameter tensor.

Fixed Diagonal as a Special Case

In a highly symmetric special case: if all s_i are initialised to the same value and the gradients have the same magnitude for all parameters at every step, then $\mathbf{s}$ stays uniform and mean-centering keeps it at zero. In that case $e^{s_i} = 1$ for all i and $v = \xi \|\mathbf{l}\|^2$, which reduces exactly to the fixed diagonal variant with $\gamma = \mathbb{1}_N$. The learnable diagonal metric therefore generalises fixed diagonal by learning *relative* per-parameter scalings, while mean-centering intentionally removes the global scale mode (which is degenerate with ξ). To recover a learnable global scale, disable mean-centering or make ξ itself learnable.

5.4 Algorithm

Algorithm 4 Learnable diagonal metric (plain embedding)

Require: all hyperparameters from Algorithm 1, plus $\mu, \lambda, s_{\max}$

```

1: Initialise  $t \leftarrow 0, \mathbf{m}_0 \leftarrow \mathbf{0}, \bar{v}_0 \leftarrow 0, \mathbf{s} \leftarrow \mathbf{0}$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $\mathbf{l}_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_{t-1})$ 
5:    $v_t \leftarrow \xi \sum_k e^{s_k} l_{t,k}^2$ 
6:    $\bar{v}_t \leftarrow \beta \bar{v}_{t-1} + (1 - \beta) v_t; \quad \hat{v}_t \leftarrow \bar{v}_t / (1 - \beta^t); \quad r_t \leftarrow 1 / (1 + |\hat{v}_t|)$ 
7:    $\mathbf{m}_t \leftarrow \beta_m \mathbf{m}_{t-1} + (1 - \beta_m) \mathbf{l}_t; \quad \hat{m}_t^i \leftarrow e^{s_i} m_{t,i} / (1 - \beta_m^t)$ 
8:    $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{\mathbf{m}}_t - \eta \lambda_{\theta} \theta_{t-1}$  // parameter update
9:    $s_i \leftarrow s_i + \mu \xi e^{s_i} l_{t,i}^2 - \mu \lambda s_i$  for each  $i$  // metric gradient ascent
10:   $s_i \leftarrow s_i - \bar{s}$  per leaf // mean-centre (gauge fix)
11:   $\mathbf{s} \leftarrow \text{clip}(\mathbf{s}, -s_{\max}, s_{\max})$ 
12: end while
13: return  $\theta_t$ 
```

5.5 Implementation

- JAX/Optax: `optimisers/jax_learnable_diag.py`
- PyTorch: `optimisers/torch_learnable_diag.py`
- Equivalence check: `optimisers/compare_learnable_diag.py`

The plain variant is `custom_sgd_learnable_diag` (sweep: `sgd_learn_diag`). The log-loss variant is `custom_sgd_log_learnable_diag` (sweep: `sgd_learn_diag_log`).

6 Off-Diagonal Metric

All preceding variants use an ambient metric h whose off-diagonal blocks (between the N parameter dimensions and the single loss dimension) are zero. We now explore what happens when those blocks are nonzero—a direction explicitly suggested by [1].

6.1 Derivation

Consider a more general ambient bilinear form

$$h = \begin{pmatrix} \gamma & \mathbf{a} \\ \mathbf{b}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1) \times (N+1)}$$

where $\mathbf{a}, \mathbf{b} \in \mathbb{R}^N$. Intuitively, the vectors $\mathbf{a}$ and $\mathbf{b}$ introduce off-diagonal couplings between the parameter space and the loss function dimension. In general, if $\mathbf{a} \neq \mathbf{b}$ this h is not symmetric, so here we simply treat it as a preconditioning matrix rather than a Riemannian metric.

Stability note. If the effective preconditioner is non-symmetric ($\mathbf{a} \neq \mathbf{b}$), the update direction need not be a descent direction even for convex objectives. To see why, note that a preconditioned gradient step $\delta\theta = -\eta P \nabla \mathcal{L}$ is a descent direction only if $\nabla \mathcal{L}^\top P \nabla \mathcal{L} > 0$, which is guaranteed when P is positive-definite (or has positive-definite symmetric part). A non-symmetric h can produce a non-symmetric $P = g^{-1}$, for which the quadratic form $\nabla \mathcal{L}^\top P \nabla \mathcal{L}$ can be negative—meaning the update moves *uphill*. We therefore

recommend defaulting to the symmetric case $\mathbf{a} = \mathbf{b}$; if using $\mathbf{a} \neq \mathbf{b}$, consider symmetrising the effective preconditioner or otherwise enforcing positive-definiteness.

The induced bilinear form on the parameter space is

$$\begin{aligned} g_{ij} &= h_{\alpha\beta} \frac{\partial \phi^\alpha}{\partial \theta^i} \frac{\partial \phi^\beta}{\partial \theta^j} \\ &= \gamma_{ij} + \xi(a_i l_j + b_j l_i + l_i l_j). \end{aligned}$$

To compute the inverse g^{ij} , we notice that g can be expressed as a rank-2 update to γ : for $U = [\mathbf{a}, \mathbf{l}]$, $V = [\mathbf{l}, \mathbf{b} + \mathbf{l}] \in \mathbb{R}^{N \times 2}$ we have

$$\begin{aligned} \gamma_{ij} + \xi U_{ik} V_{jk} &= \gamma_{ij} + \xi(a_i l_j + (b_j + l_j) l_i) \\ &= \gamma_{ij} + \xi(a_i l_j + b_j l_i + l_i l_j) \\ &= g_{ij}. \end{aligned}$$

Applying the Sherman-Morrison-Woodbury formula to $g = \gamma + \xi UV^\top$:

$$g^{ij} = \gamma^{ij} - \xi \gamma^{ik} U_{k\alpha} \left(\delta_{\alpha\beta} + \xi V_{m\alpha} \gamma^{mn} U_{n\beta} \right)^{-1} V_{l\beta} \gamma^{lj}.$$

For convenience we define the scalar contractions

$$c_{al} := a_n l^n, \quad c_{ll} := l_n l^n, \quad c_{ba} := b_n a^n, \quad c_{bl} := b_n l^n,$$

so that the 2×2 core matrix is

$$\mathbb{1}_2 + \xi V^\top \gamma^{-1} U = \begin{pmatrix} 1 + \xi c_{al} & \xi c_{ll} \\ \xi(c_{ba} + c_{al}) & 1 + \xi(c_{bl} + c_{ll}) \end{pmatrix}$$

with determinant

$$D = 1 + \xi(c_{al} + c_{bl} + c_{ll}) + \xi^2(c_{al} c_{bl} - c_{ll} c_{ba}). \quad (6)$$

Inverting the 2×2 system and contracting yields the parameter update

$$\delta \theta^i = -\eta g^{ij} l_j = -\eta \left[\frac{1 + \xi c_{al}}{D} l^i - \frac{\xi c_{ll}}{D} a^i \right]. \quad (7)$$

The closed-form inverse requires $D \neq 0$ (equivalently, $\det(\mathbb{1}_2 + \xi V^\top \gamma^{-1} U) \neq 0$). The JAX implementation includes a near-singular fallback; the PyTorch implementation solves the 2×2 system directly via `torch.linalg.solve`.

A numerical check of this has been performed in `documents/off-diag-check.ipynb`: for random $\mathbf{a}, \mathbf{b}, \gamma, \mathbf{l}$ the closed-form $g^{ij} l_j$ matches `torch.linalg.solve(g, l)` within floating-point tolerance.

Sanity Checks

Diagonal recovery ($\mathbf{a} = \mathbf{b} = \mathbf{0}$): $c_{al} = c_{ba} = c_{bl} = 0$, so $D = 1 + \xi c_{ll}$. Equation (7) gives $\delta \theta^i = -\eta l^i / (1 + \xi c_{ll})$, which matches the fixed diagonal case. ✓

Symmetric coupling ($\mathbf{a} = \mathbf{b} = \mathbf{l}$): $c_{al} = c_{bl} = c_{ba} = c_{ll}$, so $D = 1 + 3\xi c_{ll} + \xi^2(c_{ll}^2 - c_{ll}^2) = 1 + 3\xi c_{ll}$. The update simplifies to $\delta \theta^i = -\eta l^i / (1 + 3\xi c_{ll})$. ✓

One-sided coupling ($\mathbf{a} = \mathbf{0}, \mathbf{b} \neq \mathbf{0}$): $c_{al} = c_{ba} = 0$, $D = 1 + \xi(c_{bl} + c_{ll})$. The update reduces to $\delta \theta^i = -\eta l^i / D$, i.e. a scaled gradient step whose denominator depends on $\mathbf{b}$ through c_{bl} .

The update direction (7) has two components: one along l^i (the gradient direction) and one along a^i (the off-diagonal coupling direction). Depending on the choices of $\mathbf{a}$ and $\mathbf{b}$, this can lead to a variety of behaviours:

- If $\mathbf{a} = \mathbf{0}$ or $\mathbf{a} \propto \mathbf{l}$: plain gradient descent with an adaptive learning rate.
- If $\mathbf{a} = \text{momentum}$: a momentum-like update with an adaptive learning rate.
- If $\mathbf{a} = \boldsymbol{\theta}$ (the parameter vector): gradient descent combined with a form of geometry-induced weight decay or growth, depending on the sign of $\xi c_{ll}/D$.

6.2 Algorithm

Algorithm 5 Off-diagonal metric

Require: learning rate η , momentum decay β_m , metric strength ξ , weight decay λ_θ , inverse base metric γ^{-1} , mode choices for $\mathbf{a}, \mathbf{b}, \mathbf{l}, \mathbf{u}$

- 1: Initialise $t \leftarrow 0, \mathbf{m}_0 \leftarrow \mathbf{0}$
- 2: **while** θ_t not converged **do**
- 3: $t \leftarrow t + 1$
- 4: $\mathbf{g}_t \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}(\theta_{t-1})$
- 5: $\mathbf{m}_t \leftarrow \beta_m \mathbf{m}_{t-1} + (1 - \beta_m) \mathbf{g}_t$; $\hat{\mathbf{m}}_t \leftarrow \mathbf{m}_t / (1 - \beta_m^t)$
- 6: Select $\mathbf{a}, \mathbf{b}, \mathbf{l}, \mathbf{u}$ from modes (gradient, momentum, $\boldsymbol{\theta}$, or $\mathbf{0}$)
- 7: $\mathbf{U} \leftarrow (\mathbf{a}, \mathbf{l})$; $\mathbf{V} \leftarrow (\mathbf{l}, \mathbf{b} + \mathbf{l})$
- 8: Compute $c_{al}, c_{ll}, c_{ba}, c_{bl}$ and D via (6)
- 9: Compute $G^{-1}\mathbf{u}$ via Woodbury (2×2 solve + $O(N)$ vector ops)
- 10: $\theta_t \leftarrow \theta_{t-1} - \eta (G^{-1}\mathbf{u}) - \eta \lambda_\theta \theta_{t-1}$
- 11: **end while**
- 12: **return** θ_t

Note that Algorithm 5 does not use a denominator EMA (β): the full 2×2 Woodbury solve is applied directly at each step.

6.3 Implementation

- JAX/Optax: `optimisers/jax_offdiag.py`
- PyTorch: `optimisers/torch_offdiag.py`
- Equivalence check: `optimisers/compare_offdiag.py`

Sweep names follow the pattern `sgd_offdiag_{a}_{b}` where $\{\mathbf{a}\}$ and $\{\mathbf{b}\}$ are drawn from 0 (zero), 1 (gradient), m (momentum), theta (parameters). All pairs except `sgd_offdiag_0_0` (which reduces to the fixed diagonal variant) are registered in `parameters/optimizer_registry.py`.

Computational cost. Taking $\gamma_{ij} = \gamma \delta_{ij}$ (scalar base metric) and writing

$$G = \gamma \mathbb{1}_N + \xi U V^\top, \quad U = (\mathbf{a}, \mathbf{l}), \quad V = (\mathbf{l}, \mathbf{b} + \mathbf{l}),$$

the Sherman-Morrison-Woodbury formula gives

$$G^{-1}\mathbf{u} = \frac{1}{\gamma}\mathbf{u} - \frac{\xi}{\gamma^2} U \left(\mathbb{1}_2 + \frac{\xi}{\gamma} V^\top U \right)^{-1} V^\top \mathbf{u}.$$

This costs $O(N)$: only inner products and a single 2×2 matrix inversion are required, never a full $N \times N$ matrix.

Modes for $\mathbf{l}$, $\mathbf{a}$, and $\mathbf{b}$. The vector $\mathbf{l}$ encodes the direction along which the loss coordinate stretches, with two built-in choices:

- **Gradient mode:** $\mathbf{l}_i = \partial_i \mathcal{L}$ — the metric is shaped by the instantaneous slope.
- **Momentum mode:** $\mathbf{l}_i = \mathbf{m}_i$ — the metric incorporates accumulated gradient history.

The off-diagonal coupling vectors $\mathbf{a}$ and $\mathbf{b}$ each support:

- **Zero:** $\mathbf{a}_i = 0$ or $\mathbf{b}_i = 0$ — eliminates one branch of the coupling.
- **Gradient:** aligned with $\partial_i \mathcal{L}$.
- **Momentum:** aligned with $\mathbf{m}_i$.
- **Parameter:** $\mathbf{a}_i = \theta^i$ or $\mathbf{b}_i = \theta^i$ — couples geometry to position.
- **Same-as- $\mathbf{a}$:** $\mathbf{b}_i = \mathbf{a}_i$ — symmetric off-diagonal structure.

For full generality the implementations also accept user-provided vectors or callables.

7 Shared Implementation Details

7.1 Update Direction vs. Metric Direction

In all implementations, the goal is to realise

$$\delta\theta^i = -\eta g^{ij} u_j,$$

where u_j is the *update direction* and $\mathbf{l}_i$ is the direction used to *construct the metric*. These need not be the same vector. (We write $\mathbf{u}$ for the update direction to avoid confusion with the scalar damping factor r_t used in Sections 3–5.) In ordinary gradient descent one would take $u_j = l_j = \partial_j \mathcal{L}$, but we also test momentum-based choices. The four combinations are:

- $\mathbf{l} = \nabla \mathcal{L}$, $\mathbf{u} = \nabla \mathcal{L}$: pure geometric gradient descent.
- $\mathbf{l} = \nabla \mathcal{L}$, $\mathbf{u} = \mathbf{m}$: metric shaped by the instantaneous gradient, step along momentum.
- $\mathbf{l} = \mathbf{m}$, $\mathbf{u} = \nabla \mathcal{L}$: metric shaped by accumulated history, step along raw gradient.
- $\mathbf{l} = \mathbf{m}$, $\mathbf{u} = \mathbf{m}$: both metric and step are momentum-based.

7.2 Computational Cost

All variants are $O(N)$ per step (assuming scalar or diagonal γ) and never materialise a full $N \times N$ matrix:

Variant	Dominant operation	Additional metric state
Fixed diagonal	scalar $\times$ vector	none
Learnable scalar	scalar $\times$ vector	1 float (s)
Learnable diagonal	element-wise vector $\times$ vector	N floats ($\mathbf{s}$)
Off-diagonal	inner products + 2×2 inverse	none

The metric-learning overhead for the learnable variants is negligible compared to the forward and backward passes.

7.3 Shared Hyperparameters

The following hyperparameters are common to the fixed diagonal, learnable scalar, and learnable diagonal variants:

- η (**lr**): base learning rate.
- β_m (**momentum**): EMA decay for the momentum buffer. (The paper [1] uses μ for this quantity; we write β_m to avoid collision with the metric learning rate μ .)
- ξ (**xi**): strength of the induced metric correction.
- β (**beta**): EMA decay for the denominator estimate $\hat{v}$.
- λ_θ (**weight_decay**): decoupled weight decay (AdamW style).

The off-diagonal variant uses η , β_m , ξ , and λ_θ but does not use the denominator EMA β . The learnable variants additionally require:

- μ (**metric_lr**): step size for the online metric update.
- λ (**metric_reg**): L2 regularisation on s or $\mathbf{s}$.
- $s_{\max}$ (**metric_clip**): log-domain clipping bound on s , constraining $\alpha = e^s \in [e^{-s_{\max}}, e^{s_{\max}}]$.

8 Results

All optimisers are evaluated together on the same suite of benchmark tasks.

Results are compared head-to-head in the following notebooks:

- `analysis/small_examples.ipynb` — 2D synthetic functions (Beale, Rosenbrock, Himmelblau, Ackley, Rastrigin).
- `analysis/mnist_analysis.ipynb` — MNIST MLP (sweeps in progress).
- `analysis/cifar_analysis.ipynb` — CIFAR-10 ResNet-18 (sweeps in progress).
- `analysis/regression_analysis.ipynb` — Synthetic regression (sweeps in progress).
- `analysis/shake_analysis.ipynb` — Shakespeare language modelling (sweeps in progress).

Future work: for each variant, include a plot of effective step size $|\delta\theta^i|$ vs. gradient magnitude $|l_i|$, analogous to Figure 2 of [1], to visualise the gradient-clipping behaviour across variants.

Appendix A Repository Structure

A.1 Directory Layout

Directory	Contents
<code>optimisers/</code>	JAX and PyTorch optimiser implementations and equivalence checks
<code>parameters/</code>	Hyperparameter sweep scripts and shared training utilities
<code>analysis/</code>	Jupyter notebooks for loading and visualising sweep results
<code>documents/</code>	This document and supplementary notebooks
<code>images/</code>	Figures used in [1] and this document
<code>results/</code>	Local sweep results (JSON, gitignored)
<code>data/</code>	Local data files (gitignored)

A.2 Running Sweeps

Hyperparameter sweeps are driven by scripts in `parameters/` and support two backends: **local** (Optuna + JSON files, no account required) and **WandB** (cloud logging).

Script	Task	Model
<code>sweep_mnist_mlp.py</code>	MNIST classification	MLP (64 hidden)
<code>sweep_cifar10_resnet18.py</code>	CIFAR-10 classification	ResNet-18
<code>sweep_regression.py</code>	Synthetic regression	MLP
<code>sweep_shake.py</code>	Shakespeare language model	MiniGPT
<code>sweep_small_examples.py</code>	2D test functions	n/a

All scripts share a common CLI:

- `--optimiser`: optimiser name (see below).
- `--num_runs`: number of Optuna trials (default: 50).
- `--backend`: local or wandb (default: wandb).
- `--index`: run index for organising multiple runs of the same sweep.

Results are written to `results/<task>/<optimiser>/run_<index>/<run_id>.json`, each containing the full training history, final summary metrics, and the hyperparameter config used.

A.3 Available Optimisers

Sweep name	Description
<code>adam</code> , <code>adamw</code> , <code>sgd</code> , <code>muon</code>	Baselines
<code>sgd_metric</code> , <code>sgd_log_metric</code> , <code>sgd_rms</code>	Fixed diagonal metric
<code>sgd_learn_scalar</code> , <code>sgd_learn_scalar_log</code>	Learnable scalar metric
<code>sgd_learn_diag</code> , <code>sgd_learn_diag_log</code>	Learnable diagonal metric
<code>sgd_offdiag_{a}_{b}</code>	Off-diagonal metric ($\{0, l, m, \theta\}^2$ pairs, excluding 0.0)

The full optimiser registry is `parameters/optimizer_registry.py`.

A.4 Analysis Notebooks

Notebook	Task
<code>analysis/small_examples.ipynb</code>	2D test functions (Beale, Rosenbrock, etc.)
<code>analysis/mnist_analysis.ipynb</code>	MNIST MLP
<code>analysis/cifar_analysis.ipynb</code>	CIFAR-10 ResNet-18
<code>analysis/regression_analysis.ipynb</code>	Synthetic regression
<code>analysis/shake_analysis.ipynb</code>	Shakespeare language modelling

Each notebook supports both backends. Configure `BACKEND`, `TASK_TAG`, and `OPTIMIZERS` in the second cell. Outputs include training curves, 2D histograms of best metric vs. epoch, time-to-best analysis, and best-hyperparameter tables (also saved as CSV to `analysis/`).

References

- [1] T. R. Harvey, “The Optimiser Hidden in Plain Sight: Training with the Loss Landscape’s Induced Metric,” *arXiv preprint arXiv:2509.03594*, 2025.

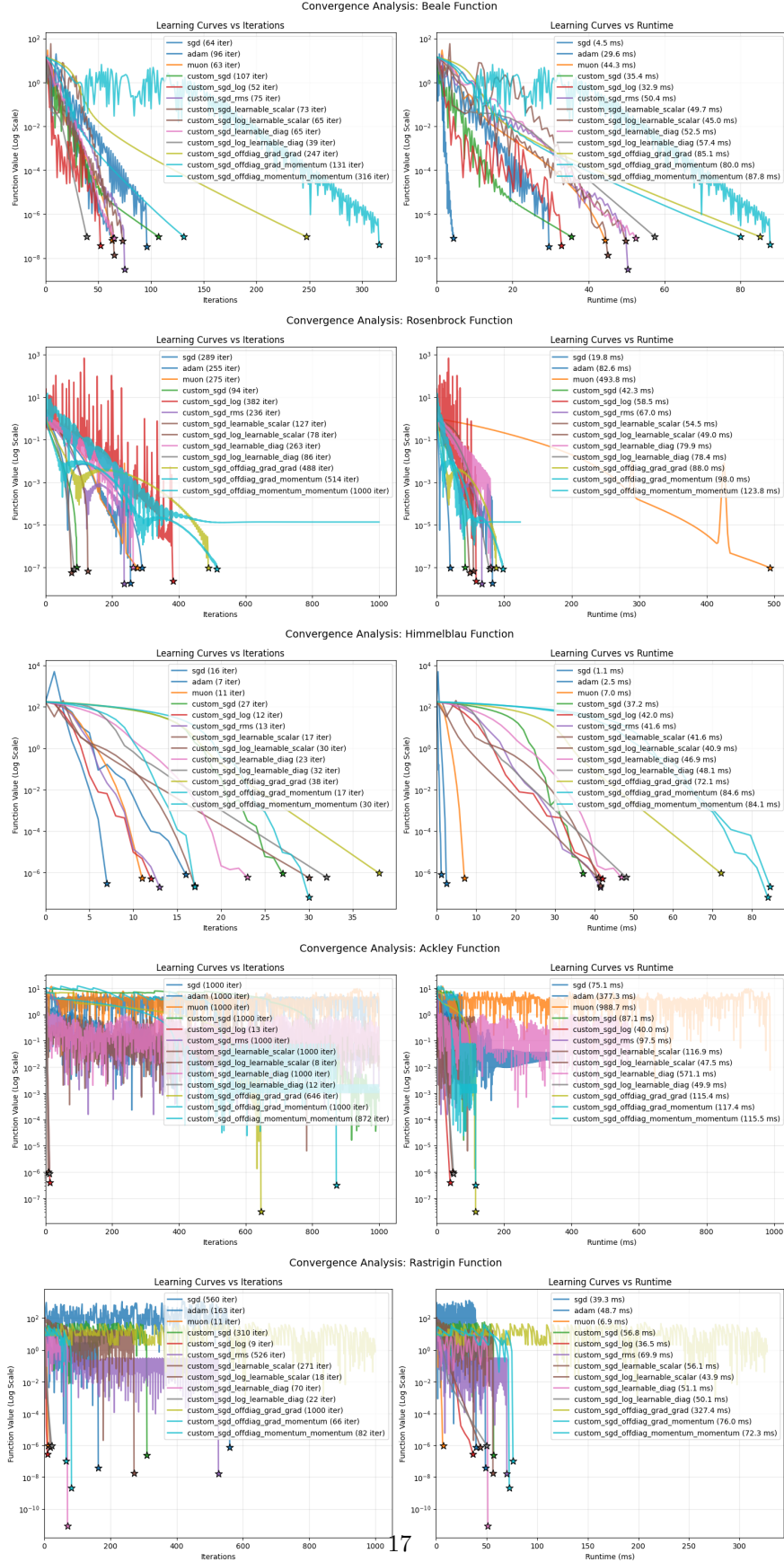


Figure 1: Small-example benchmark results for all optimiser variants. Each plot shows the best hyperparameter configuration found by the sweep. (*Fixed and off-diagonal sweeps are complete for small examples.*)